Testing Strategy and Test Results

Introduction

This document outlines the testing strategy, test cases, and testing results for the backend part of the application. The primary goal of testing was to ensure that all controllers, services, and routes behave as expected with little to no bugs

The testing process focused on:

- Verifying correct functionality
- Validating error handling
- Improving overall code coverage

The testing framework used was:

- Jest for unit testing and mocking

Testing Strategy

Unit testing was used to test individual functions in isolation without depending on external systems such as databases.

The following dependencies were mocked:

- Database model methods
- Express request and response objects

Examples of database mocked methods:

- findById
- find
- findOne
- create
- findByIdAndUpdate

This approach allowed the tests to run quickly, produce predictable results and test all possible branches and paths independently. Special attention was given to covering all logical branches in the code. Each controller function was tested for successful execution, missing request data, authentication failures, missing database records and edge cases to ensure high branch and statement coverage.

The backend testing process successfully validated the application's core functionality, error handling, and authorization logic.

The use of unit testing and mocking enabled reliable and isolated testing of all major components. High coverage was achieved by thoroughly testing normal operations, edge cases, and failure

scenarios. The implemented tests improve reliability and maintainability. They further helped to catch bugs early in the development process.

The screenshot below shows the code coverage results generated after running the automated test suite using the testing framework. The report provides a summary of statement coverage, branch coverage, function coverage, and line coverage across the application. The high coverage percentages indicate that most parts of the system, including validation logic, error handling, conditional branches, and successful execution paths, were tested thoroughly. This demonstrates that the implemented unit tests effectively verified the reliability and correctness of the backend functionality.